算法基础：习题解答 5

问题 1：树的最长非平凡路径

若 $|V| < 2$，则树中不存在非平凡路径；以下假设 $|V| \geq 2$。

这个问题本质上是在求树的直径。任选一个节点作为根，将原树看作一棵有根树。对于树上每一个节点 u，维护从 u 出发向下进入其子树的最长路径长度和次长路径长度，分别记为 $d_1(u)$ 和 $d_2(u)$。这两条路径分别进入不同的子树，缺失的方向视为长度为 0 的空路径。经过 u、在两个向下方向间连接的最长路径长度为

$$d_1(u) + d_2(u).$$

因此整棵树的直径长度为

$$\max_{u \in V} \{d_1(u) + d_2(u)\}.$$

其中 V 表示树的节点集合，E 表示树的边集合。

为了计算这些值，我们对树进行一次深度优先搜索。设 u 的儿子集合为 $\mathrm{child}(u)$。对于每个儿子 v，从 u 经过 v 向下走能得到的最长路径长度为 $d_1(v) + 1$。因此 $d_1(u)$ 和 $d_2(u)$ 分别是多重集合

$$\{d_1(v) + 1 : v \in \mathrm{child}(u)\}$$

中的最大值和次大值。如果集合为空或只有一个元素，则缺失的值记为 0。

为构造具体路径，同时维护对应的终点：$\mathrm{end}_1(u)$ 是最长向下路径的终点，$\mathrm{end}_2(u)$ 是次长向下路径的终点。在 DFS 中维护全局变量

$$ans = \max_{u \in V} \{d_1(u) + d_2(u)\}$$

和取得该最大值的节点 u^*。最终令

$$A = \mathrm{end}_1(u^*), \qquad B = \mathrm{end}_2(u^*).$$

树中任意两点间存在唯一简单路径，因此 A 到 B 的路径就是一条直径路径。第一段算法求出两个端点；第二段算法从 A 出发进行 DFS，以回溯方式恢复 A 到 B 的完整路径。

Algorithm 1 求树的直径

任选一个节点 r 作为根
$ans \leftarrow 0$, $u^* \leftarrow r$
procedure DFS(u, p)
    $d_1(u) \leftarrow 0$, $d_2(u) \leftarrow 0$
    $\text{end}_1(u) \leftarrow u$, $\text{end}_2(u) \leftarrow u$
    for each $v \in \text{Adj}(u)$ **do**
        if $v = p$ **then**
            continue
        end if
        DFS(v, u)
        $x \leftarrow d_1(v) + 1$
        if $x > d_1(u)$ **then**
            $d_2(u) \leftarrow d_1(u)$, $\text{end}_2(u) \leftarrow \text{end}_1(u)$
            $d_1(u) \leftarrow x$, $\text{end}_1(u) \leftarrow \text{end}_1(v)$
        else if $x > d_2(u)$ **then**
            $d_2(u) \leftarrow x$, $\text{end}_2(u) \leftarrow \text{end}_1(v)$
        end if
    end for
    if $d_1(u) + d_2(u) > ans$ **then**
        $ans \leftarrow d_1(u) + d_2(u)$, $u^* \leftarrow u$
    end if
end procedure
DFS(r, NIL)

Algorithm 2 恢复 A 到 B 的路径

$A \leftarrow \text{end}_1(u^*)$, $B \leftarrow \text{end}_2(u^*)$, $P \leftarrow$ empty list
procedure DFS-PATH(u, p)
    P.push_back(u)
    if $u = B$ **then**
        return true
    end if
    for each $v \in \text{Adj}(u)$ **do**
        if $v \neq p$ **then**
            if DFS-PATH(v, u) **then**
                return true
            end if
        end if
    end for
    P.pop_back()
    return false
end procedure
DFS-PATH(A, NIL)
return P

正确性证明

对于任意节点 u，$d_1(u)$ 和 $d_2(u)$ 表示从 u 向下进入两个不同子树方向的最长和次长路径长度。因此将它们在 u 处连接得到一条简单路径，长度为 $d_1(u) + d_2(u)$，所以算法的 ans 是某条树上简单路径的长度。

另一方面，考虑任意简单路径 P。将树以 r 为根，设 a 为 P 上距离根最近的节点，即 P 两端点的最低公共祖先。路径 P 从 a 出发分别进入两个不同子树方向，或者其中一个方向为空。因此 P 的长度不超过 $d_1(a) + d_2(a)$，进而不超过 $\max_{u \in V}(d_1(u) + d_2(u))$。

综上，算法计算的 ans 正好等于树的直径长度。算法记录了对应的两个端点 A, B，而树中两点间的简单路径唯一，故第二段 DFS 恢复出的路径就是一条最长非平凡路径。

时间复杂度

两段 DFS 均至多访问每个节点一次，每条无向边检查常数次，总时间为 $O(|V|+|E|)$。

问题 2：判断图是否可以 2 染色

给定无向图 $G=(V,E)$，判断其是否可以被 2 染色，等价于判断 G 是否为二分图。

我们使用 BFS 染色。对每个尚未染色的节点 s，将其染成红色，然后从 s 开始 BFS。访问边 (u,v) 时，若 v 未染色，则将 v 染为与 u 相反的颜色；若 v 已染色，则检查两端颜色是否相同。如果相同，则存在冲突，图不能 2 染色。若整个过程没有冲突，则得到合法的染色方案。

Algorithm 3 判断图是否可以 2 染色

```
for each u ∈ V do
    color(u) ← NIL
end for
for each s ∈ V do
    if color(s) ≠ NIL then
        continue
    end if
    color(s) ← 0, Q ← empty queue
    Q.push(s)
    while Q is not empty do
        u ← Q.pop()
        for each v ∈ Adj(u) do
            if color(v) = NIL then
                color(v) ← 1 − color(u)
                Q.push(v)
            else if color(v) = color(u) then
                return No
            end if
        end for
    end while
end for
return color(·)
```

正确性证明

若算法返回一个染色方案，则每条边都已检查过：若 v 在访问 u 时首次染色，其颜色与 u 相反；若 v 已染色，则算法也检查了两端颜色不同。因此返回的方案是合法的 2 染色。

反之，若算法返回 No，则存在边 (u,v) 两端颜色相同。同一 BFS 分量中，节点颜色等于它到起点的 BFS 树距离的奇偶性。于是 u,v 到起点的树距离具有相同奇偶性，将两条树路径与边 (u,v) 合并，得到一个奇长度闭合走法，从中可提取奇环。含奇环的图不能 2 染色，因此返回 No 时图确实不可 2 染色。

时间复杂度

每个节点访问一次，每条无向边检查常数次，时间为 $O(|V|+|E|)$。

问题 3：有界正整数边权的最短路

给定有向图 $G=(V,E)$，每条边的权重属于 $\{1,2,\ldots,w\}$，给定源点 $s\in V$，要求在 $O(w(|V|+|E|))$ 时间内求出 s 到所有节点的最短距离。

这可以看作 BFS 的推广。普通 BFS 所有边权为 1，每次从当前距离层扩展到下一层；边权最大为 w 时，从距离为 D 的节点出发，只可能更新距离为 $D+1,\ldots,D+w$ 的节点。

维护 $w+1$ 个队列 $B_0,B_1,\ldots,B_w$。当节点距离更新为 x 时，将其放入 $B_{x \bmod (w+1)}$。按 $D=0,1,2,\ldots$ 处理 $B_{D \bmod (w+1)}$；处理完第 D 层后再进入第 $D+1$ 层。队列保留过期条目，弹出时用当前距离判断并跳过。

Algorithm 4 有界正整数边权的最短路

```
for each v ∈ V do
    dist(v) ← +∞
end for
for i = 0 to w do
    B_i ← empty queue
end for
dist(s) ← 0, B_0.push(s)
for D = 0 to w(|V| − 1) do
    i ← D mod (w + 1)
    while B_i ≠ ∅ do
        u ← B_i.pop()
        if dist(u) ≠ D then
            continue
        end if
        for each outgoing edge (u, v) with weight c do
            if dist(v) > D + c then
                dist(v) ← D + c
                B_{dist(v) mod (w+1)}.push(v)
            end if
        end for
    end while
end for
return dist(·)
```

正确性证明

记 $\delta(v)$ 为从 s 到 v 的真实最短距离。证明：处理第 D 层时，所有满足 $\delta(v) < D$ 的节点都已正确处理；若节点 u 在第 D 层被有效处理，即弹出时 $\mathrm{dist}(u) = D$，则 $D = \delta(u)$。

用归纳法。$D = 0$ 时，只有源点 s 的距离为 0。假设所有小于 D 的距离层已正确处理。第 D 层弹出 u 且 $\mathrm{dist}(u) = D$ 时，有 $\delta(u) \le D$，因为 $\mathrm{dist}(u)$ 总由某条实际路径松弛得到。若 $\delta(u) < D$，由归纳假设 u 已在更早层被正确处理，当前条目应当过期，不可能满足 $\mathrm{dist}(u) = D$。所以 $\delta(u) = D$。

节点 u 被有效处理时，算法用其所有出边松弛。设 v 满足 $\delta(v) = D' \le w(|V| - 1)$，取一条最短路，最后一条边为 (u, v)，边权为 c。则 $\delta(u) = D' - c < D'$。由归纳假设，u 在 $\delta(u)$ 层被处理，将 v 的距离松弛为 $\delta(u) + c = \delta(v)$ 并放入对应桶，最终在 $\delta(v)$ 层处理 v。故所有可达点的最短距离均能正确求出，不可达点保持 $+\infty$。

由于所有新条目的距离处于 $D + 1$ 到 $D + w$，而桶数为 $w + 1$，处理当前桶时不会误弹出未来距离层的有效条目。

时间复杂度

正权最短路可取为简单路径，至多包含 $|V| - 1$ 条边，因此有限最短距离不超过 $w(|V| - 1)$，外层扫描 $O(w|V|)$ 次。令

$$V_D = \{u \in V : \delta(u) = D\}.$$

第 D 层只有效处理 V_D 中的节点及其出边，有效处理代价为

$$O\big(|V_D| + |\{(u, v) \in E : u \in V_D\}|\big).$$

每条有向边只松弛一次，每次成功松弛至多产生一个过期条目，全部过期条目的处理代价为 $O(|E|)$。于是总时间为

$$\sum_{D=0}^{w(|V|-1)} O\big(|V_D| + |\{(u, v) \in E : u \in V_D\}| + 1\big) + O(|E|) = O(w|V| + |E|),$$

从而满足所要求的 $O(w(|V| + |E|))$ 时间界。

问题 4：最大收益比有向环

给出一个加性 ε 近似算法，即返回的有向环 C 满足

$$\rho(C) := \frac{\sum_{e\in C} f(e)}{\sum_{v\in C} g(v)} \geq \mathrm{OPT} - \varepsilon,$$

其中 OPT 为最优收益比。假设所有 $g(v) > 0$；若允许 $g(v) = 0$，需要先单独处理零代价有向环。

考虑二分答案 λ。对于给定的 λ，问题变为判断是否存在有向环 C 满足

$$\rho(C) > \lambda \quad\Longleftrightarrow\quad \sum_{e\in C} f(e) - \lambda \sum_{v\in C} g(v) > 0.$$

定义新边权

$$w_\lambda(u,v) = \lambda g(v) - f(u,v).$$

任意有向环 C 满足

$$\sum_{(u,v)\in C} w_\lambda(u,v) = \lambda \sum_{v\in C} g(v) - \sum_{e\in C} f(e).$$

因此，存在收益比大于 λ 的环，当且仅当新图中存在负权环。使用 Bellman–Ford 可以在 $O(|V||E|)$ 时间内判定并恢复负权环。

设图中至少存在一个有向环。每个环的收益比可写成各边 $f(u,v)/g(v)$ 的加权平均：

$$\rho(C) = \sum_{(u,v)\in C} \frac{g(v)}{\sum_{x\in C} g(x)} \frac{f(u,v)}{g(v)}.$$

故可取初始上下界

$$L_0 = \min_{(u,v)\in E} \frac{f(u,v)}{g(v)}, \qquad R_0 = \max_{(u,v)\in E} \frac{f(u,v)}{g(v)}.$$

若收益均非负，也可取 $L_0 = 0$ 和

$$R_0 = \frac{\max_{e\in E} f(e)}{\min_{v\in V} g(v)}.$$

给定 $\varepsilon > 0$，当 $R_0 > L_0$ 时令

$$T = \max\left\{0, \left\lceil \log_2 \frac{R_0 - L_0}{\varepsilon} \right\rceil\right\};$$

当 $R_0 = L_0$ 时令 $T = 0$。二分 T 轮后区间长度至多为 ε。每轮调用一次负环检测，因此总时间为

$$O\left(|V| + |E| + |V||E| \log\left(1 + \frac{R_0 - L_0}{\varepsilon}\right)\right).$$

在非负收益、采用 $[0, R_0]$ 时，通常写为 $O(|V||E| \log(R_0/\varepsilon))$，并另计初始化时间及零轮情形。

Algorithm 5 最大收益比有向环的加性近似

若 G 无有向环，**return** No Cycle；否则令 ans 为任意有向环
$L \leftarrow L_0$, $R \leftarrow R_0$
for $t = 1$ to T **do**
    $\lambda \leftarrow (L + R)/2$
    for each edge $(u, v) \in E$ **do**
        $w_\lambda(u, v) \leftarrow \lambda g(v) - f(u, v)$
    end for
    $(ok, C) \leftarrow$ BF-Negative-Cycle(G, w_λ)
    if $ok =$ true **then**
        $L \leftarrow \lambda$, $ans \leftarrow C$
    else
        $R \leftarrow \lambda$
    end if
end for
return ans

近似保证

当 $\lambda < \mathrm{OPT}$ 时存在负环；当 $\lambda \geq \mathrm{OPT}$ 时不存在收益比大于 λ 的环。因此二分始终保持

$$L \leq \mathrm{OPT} \leq R.$$

每次找到负环时，其收益比严格大于当前 λ，将该环保存为 ans，并令 $L = \lambda$。初始化保存的任意环收益比至少为 L_0，因此始终有 $\rho(ans) \geq L$。最终 $R - L \leq \varepsilon$，所以

$$\rho(ans) \geq L \geq \mathrm{OPT} - \varepsilon.$$

负环检测子程序

BF-Negative-Cycle 输入图和边权函数，输出布尔值 ok 和环 C。若存在负环，则 $ok =$ true 且 C 是负环；否则返回 (false, NIL)。将所有节点距离初始化为 0，等价于添加到所有节点均有零权边的超级源点，因而能检测任意连通分量中的负环。

Algorithm 6 Bellman–Ford 负环检测与恢复

```
procedure BF-Negative-Cycle(G, w)
    for each v ∈ V do
        d(v) ← 0, pre(v) ← NIL
    end for
    x ← NIL
    for i = 1 to |V| do
        x ← NIL
        for each edge (u, v) ∈ E do
            if d(v) > d(u) + w(u, v) then
                d(v) ← d(u) + w(u, v)
                pre(v) ← u, x ← v
            end if
        end for
    end for
    if x = NIL then
        return (false, NIL)
    end if
    y ← x
    for i = 1 to |V| do
        y ← pre(y)
    end for
    C ← empty list, z ← y
    repeat
        C.push_back(z)
        z ← pre(z)
    until z = y
    C.reverse()
    return (true, C)
end procedure
```

第 $|V|$ 轮仍有松弛时，沿前驱回退 $|V|$ 次可进入负环，再沿前驱遍历一周并反转节点顺序，得到按有向边方向排列的负环。

问题 5：最小权重简单有向圈

原始版本：允许负圈

若没有“无负圈”的保证，该问题是 NP-hard 的。考虑从有向哈密顿圈问题归约：给定有向图 $G = (V, E)$，令每条边权重为 -1。任意简单有向圈的权重等于其长度的相反数，因此权重最小的简单有向圈就是长度最大的简单有向圈。

若存在多项式时间算法求出最小权重简单有向圈，只需检查其权重是否为 $-|V|$，即可判断是否存在哈密顿圈。因此该问题是 NP-hard 的。

下面给出枚举算法。若图无自环，则只需枚举大小至少为 2 的点集；一般情况下也包括大小为 1 的点集，以覆盖自环。

Algorithm 7 枚举最小权重简单有向圈

```
ans ← +∞, best ← ∅
for each nonempty subset S ⊆ V do
    for each permutation (v_1, v_2, ..., v_k) of S do
        if (v_i, v_{i+1}) ∈ E for all 1 ≤ i < k and (v_k, v_1) ∈ E then
            W ← Σ_{i=1}^{k-1} w(v_i, v_{i+1}) + w(v_k, v_1)
            if W < ans then
                ans ← W, best ← (v_1, ..., v_k, v_1)
            end if
        end if
    end for
end for
if ans = +∞ then
    return No Cycle
else
    return (best, ans)
end if
```

正确性证明

每个排列的顶点互不相同，若所有相邻边及首尾边存在，就构成简单有向圈。反之，任意简单有向圈 $v_1 \to \cdots \to v_k \to v_1$ 的顶点集为 $S = \{v_1, \ldots, v_k\}$，算法枚举 S 的所有排列，必然枚举到该圈。因此算法覆盖所有简单有向圈并取权重最小者，算法正确。

时间复杂度

记 $n = |V|$。共有 $2^n - 1$ 个非空点集；若不考虑自环，则大小至少为 2 的点集有 $2^n - n - 1$ 个。每个大小为 k 的点集有 $k!$ 个排列，每个排列的检查和权重计算需要 $O(k)$ 时间。因此总时间为

$$\sum_{k=1}^{n} \binom{n}{k} k!\, O(k) = O\left(\sum_{k=1}^{n} \frac{n!}{(n-k)!}\, k \right) = O(n!\, n),$$

也满足较宽松的 $O(n!\, n^2)$ 上界。这里采用邻接矩阵在常数时间内判断边是否存在。

调整版本：图中不存在负圈

在无负圈保证下存在多项式时间算法。先用所有自环更新答案，再处理不含自环的圈。顶点编号为 $1, 2, \ldots, n$。维护 Floyd 数组 $d[i][j]$：在第 k 轮开始时，它表示只允许编号小于 k 的节点作为中转点时，从 i 到 j 的最短路径长度。

枚举 k 时，考虑所有 $i < k, j < k$。若存在边 (k, i) 和 (j, k)，则

$$k \to i \rightsquigarrow j \to k$$

构成经过 k 的有向圈，其中最短路径 $i \rightsquigarrow j$ 仅经过编号小于 k 的中转点。因此候选权重为

$$w(k, i) + d[i][j] + w(j, k).$$

先用这些候选更新答案，然后再进行以 k 为中转点的 Floyd 更新。维护路径恢复数组 $next[i][j]$，表示当前从 i 到 j 的最短路上，i 后面的第一个节点。

为保持 $O(n^3)$ 时间界，每轮先找出本轮最优的点对，只在本轮最优值改善全局答案时恢复一次路径，随后才更新 Floyd 数组。

Algorithm 8 无负圈时的最小权重简单有向圈

```
ans ← +∞, bestCycle ← ∅
for i = 1 to n do
    for j = 1 to n do
        d[i][j] ← +∞, next[i][j] ← NIL
        a[i][j] ← +∞
    end for
    d[i][i] ← 0, next[i][i] ← i
end for
for each edge (u, v) ∈ E do
    a[u][v] ← min{a[u][v], w(u, v)}
    if u = v and w(u, u) < ans then
        ans ← w(u, u), bestCycle ← [u, u]
    else if u ≠ v and w(u, v) < d[u][v] then
        d[u][v] ← w(u, v), next[u][v] ← v
    end if
end for
for k = 1 to n do
    roundBest ← +∞, (i*, j*) ← (NIL, NIL)
    for i = 1 to k − 1 do
        for j = 1 to k − 1 do
            if a[k][i], a[j][k], d[i][j] are all finite then
                W ← a[k][i] + d[i][j] + a[j][k]
                if W < roundBest then
                    roundBest ← W, (i*, j*) ← (i, j)
                end if
            end if
        end for
    end for
    if roundBest < ans then
        P ← RecoverPath(i*, j*, next)
        ans ← roundBest, bestCycle ← [k] + P + [k]
    end if
    for i = 1 to n do
        for j = 1 to n do
            if d[i][k], d[k][j] are finite and d[i][k] + d[k][j] < d[i][j] then
                d[i][j] ← d[i][k] + d[k][j]
                next[i][j] ← next[i][k]
            end if
        end for
    end for
end for
if ans = +∞ then
    return No Cycle
else
    return (bestCycle, ans)
end if
```

路径恢复

RecoverPath$(i, j, next)$ 返回从 i 到 j 的最短路。若 $i = j$，返回 $[i]$；否则从 i 开始，不断沿 $next[i][j]$ 走，直到到达 j。无负圈时，若最短走法中出现重复节点，可删除重复节点之间的有向圈而不增加长度，因此可以取简单最短路。路径数组只在距离严格改善时更新。

正确性证明

第 k 轮检查答案前，$d[i][j]$ 只允许编号小于 k 的节点作为中转点。对存在的边 $(k, i), (j, k)$，把它们与从 i 到 j 的简单最短路连接，构成编号最大顶点为 k 的简单有向圈；$i = j$ 时得到两条边组成的圈。自环已单独处理。因此每个候选都是合法圈的权重，不会小于最优值。

另一方面，取任意简单有向圈 C，设 k 是其编号最大的顶点，沿 C 从 k 出发的第一条边为 (k, i)，返回 k 的最后一条边为 (j, k)。圈上从 i 到 j 的部分仅经过编号小于 k 的顶点。因此第 k 轮考虑的候选

$$w(k, i) + d[i][j] + w(j, k)$$

不大于 C 的权重。故算法得到的答案不大于最优圈权重。两方向结合，返回的就是最小权重简单有向圈。

时间复杂度

初始化 $d, a, next$ 需要 $O(n^2)$ 时间，读入边需要 $O(|E|)$。每轮枚举 $i < k, j < k$，总时间为

$$\sum_{k=1}^{n} O(k^2) = O(n^3).$$

每轮 Floyd 更新需要 $O(n^2)$，共 $O(n^3)$。每轮至多恢复一次路径，每次 $O(n)$，总恢复代价为 $O(n^2)$。因此总时间为

$$O(n^3 + |E|),$$

空间复杂度为 $O(n^2)$。